

LLM Code Smells

Designing maintainable, reliable and robust software that integrates large language models

Md Saeed Siddik

Assistant Professor,
Institute of IT, University of Dhaka

Learning Objectives

- 1 **Distinguish** LLM code smells from bugs, prompt smells, and smells in LLM-generated code.
- 2 **Explain** the nine-smell taxonomy: Structural / API-usage, Data-semantics and Protocol smells.
- 3 **Recognize** each smell in real integration code and describe its impact on quality.
- 4 **Refactor** smelly code using design patterns such as Facade, Factory Method, Strategy and Adapter.
- 5 **Evaluate** what static analysis can and cannot detect.

THE BIG IDEA

An LLM call is a software component.

Like any component, it deserves an explicit contract: and design patterns are how we make that contract enforceable.



AI-Generated Code Smells

Bugs or poor practices in the output generated BY models.



LLM Integration Code Smells

Poor practices in human-written code that orchestrates or calls LLMs.

LLM Code Smells are code-level poor practices in human-written source code that integrates or orchestrates LLM inference within software systems.

They don't always cause immediate crashes, but they quietly undermine **maintainability, reliability, performance, and robustness.**

Why Do We Need LLM-Specific Code Smells?

TRADITIONAL COMPONENT

- Deterministic interface
- Known configuration semantics
- Predictable execution behaviour
- Stable across deployments

LLM COMPONENT

- Non-deterministic output
- Natural-language interface
- Provider- and model-specific parameters
- Behaviour changes over time: even without code changes

DESIGN CONSEQUENCE

The API call alone is not enough.

We need an explicit inference contract that names the model, instructions, output shape, sampling policy, resource limits and traceability.

Classic GoF smells still apply. These are additional smells that only exist because of how LLMs behave.

DEFINITION

What Is an LLM Code Smell?

“A recurring, low-level coding practice in human-written source code that integrates or orchestrates LLM/VLM inference and degrades software quality: without necessarily being an explicit bug.”

IT IS NOT THE SAME AS...

Smells in LLM-generated code quality of code an LLM wrote

Prompt smells quality of the prompt text itself

System-level failures hallucination, jailbreaks, RAG drift

Model-quality problems accuracy of the model

FOUR QUALITY ATTRIBUTES AT RISK

Robustness

Resilience under errors and unexpected inputs

Performance

Latency, memory, compute and monetary cost

Maintainability

Reproducibility, portability, traceability, observability

Reliability

Correctness, consistency and behavioural stability

The Nine-Smell Taxonomy

Structural / API usage

How the call is shaped and what it carries

NSM	No System Message
OSP	Overspecified Sampling Parameters
AIC	Anonymous Inference Call

Data semantics

What goes in and what comes out

NSO	No Structured Output
RVP	Raw Vision Payload

Protocol

Which configuration governs the inference

NMVP	No Model Version Pinning
TNES	Temperature Not Explicitly Set
UMM	Unbounded Max Metrics
RENES	Reasoning Effort Not Explicitly Set

LLM Code Smells



Structural

[No System Message]

[Anonymous Inference Call]*

Defects in how the API request is composed.



Data Semantic

[No Structured Output]

[Raw Vision Payload]*

Incorrect assumptions about input/output structures.



Protocol

[Unbounded Max Metrics]

[LLM Temperature Not Explicitly Set]

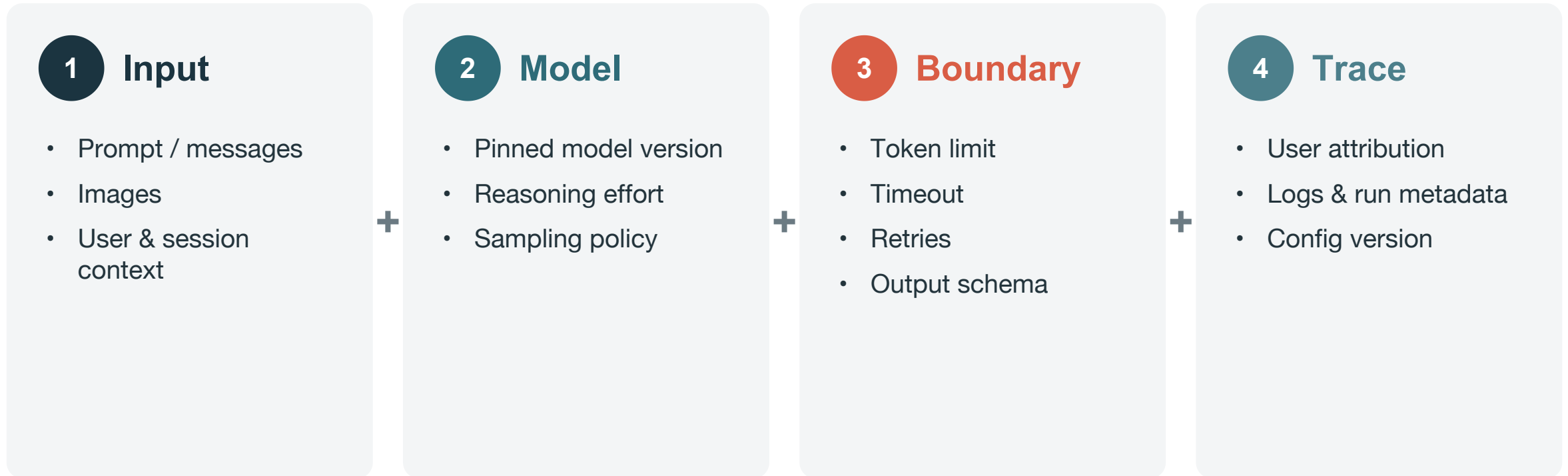
[No Model Version Pinning]

[Reasoning Effort Not Explicitly Set]*

[Overspecified Sampling Parameters]*

Unstable system-wide inference configurations.

Think of an LLM Call as a Contract



Each of the nine smells is a missing or implicit clause in this contract. Design patterns give us places to write the clauses down: once: and enforce them.

NSM: No System Message

SMELLY

```
messages = [  
    {"role": "user", "content": user_prompt}  
]  
response = client.chat.completions.create(  
    model=MODEL, messages=messages)
```

REFACTORED

```
messages = [  
    {"role": "system", "content": SYSTEM_POLICY},  
    {"role": "user", "content": user_prompt},  
]  
# SYSTEM_POLICY: versioned, reviewed, stable
```

PROBLEM

No stable global role, goals or constraints. Every behaviour depends on whatever the user typed.

REFACTOR

Treat the system-level instruction as a first-class, versioned artifact that every call receives.

DESIGN PATTERN LINK

Template Method / Strategy: a stable instruction scaffold with a variable user part.

25.3%

of the 692 studied projects contained at least one NSM occurrence.

OSP: Overspecified Sampling Parameters

SMELLY

```
response = client.generate(  
    prompt,  
    temperature=0.8,  
    top_p=0.95,  
    top_k=40,  
) # three interacting randomness controls
```

REFACTORED

```
# One primary control, with a documented rationale  
response = client.generate(  
    prompt,  
    temperature=0.2, # deterministic-ish: grading task  
)
```

PROBLEM

Multiple randomness controls interact; tuning and reproducing behaviour becomes guesswork.

REFACTOR

Use a minimal, documented sampling contract: one primary control with a stated reason.

DESIGN PATTERN LINK

Strategy: encapsulate a named decoding policy (e.g. Deterministic, Creative).

13.4%

of projects showed overspecified sampling parameters.

AIC: Anonymous Inference Call

SMELLY

```
def answer(prompt):  
    return client.chat.completions.create(  
        model=MODEL,  
        messages=[{"role": "user", "content": prompt}],  
    ) # who asked? which session?
```

REFACTORED

```
def answer(prompt, session_id):  
    return client.chat.completions.create(  
        model=MODEL,  
        messages=[{"role": "user", "content": prompt}],  
        user=pseudonymize(session_id), # traceable  
    )
```

PROBLEM

A multi-user call carries no recoverable user or session attribution.

REFACTOR

Propagate a stable, pseudonymous identifier through the wrapper into every call.

DESIGN PATTERN LINK

Context Object / Facade: request-scoped metadata travels with the call.

0.7%

of projects: rare, but it undermines auditing and incident response.

NSO: No Structured Output

SMELLY

```
response = client.responses.create(  
    model=MODEL, input=prompt)  
result = json.loads(response.output_text)  
# hopes the model returned valid JSON
```

REFACTORED

```
response = client.responses.parse(  
    model=MODEL, input=prompt,  
    text_format=GradeReport, # Pydantic schema  
)  
result: GradeReport = response.output_parsed
```

PROBLEM

Free-form text is treated as schema-compliant data: missing fields, type errors, brittle parsing.

REFACTOR

Enforce a JSON / schema contract at the API boundary and validate before use.

DESIGN PATTERN LINK

Adapter: convert the provider response into a typed domain object.

43.6%

of projects parsed
unstructured output as if it
were structured.

Data Semantic Smell: No Structured Output (NSO)

Prevalence: 43.64% of projects

✗ BAD

```
response = client.chat.completions.create(  
    model="gpt-4o-2024-11-20",  
    messages=messages  
)  
# System consumes response.text assuming it is valid JSON
```



Consuming free-form text where structured fields are expected leads to silent truncation, parsing errors, and schema drift.

✓ GOOD

```
response = client.chat.completions.create(  
    model="gpt-4o-2024-11-20",  
    response_format={  
        "type": "json_schema",  
        "json_schema": {"name": "Result", "schema": result_schema}  
    },  
    messages=messages  
)
```

Architectural Takeaway

Enforce data structure and validation before parsing. The LLM output contract must match downstream pipeline expectations strictly.

RVP: Raw Vision Payload

SMELLY

```
image_bytes = open("scan_original.png", "rb").read()
prompt = build_prompt(image_bytes)
# 12 MB, full resolution, unredacted
```

REFACTORED

```
img = load("scan_original.png")
img = resize(img, max_side=1024)
img = compress(img, quality=80)
img = redact_faces_and_ids(img)
assert size(img) < MAX_IMAGE_BYTES
prompt = build_prompt(img)
```

PROBLEM

Raw images leak private data, inflate tokens and cost, and slow every request.

REFACTOR

Preprocess and bound image payloads before inference: resize, compress, redact, limit.

DESIGN PATTERN LINK

Pipeline / Chain of Responsibility: composable input-hygiene steps.

18.9%

of projects sent raw vision payloads to the model.

NMVP: No Model Version Pinning

SMELLY

```
response = client.chat.completions.create(  
    model="gpt-4o", # floating alias  
    messages=messages)  
# provider updates silently change behaviour
```

REFACTORED

```
MODEL = "gpt-4o-2024-11-20" # immutable snapshot  
response = client.chat.completions.create(  
    model=MODEL,  
    messages=messages)  
log_run(model=MODEL, config_version=CFG_VERSION)
```

PROBLEM

Provider-side updates change behaviour without any change in your source code.

REFACTOR

Pin an immutable model identifier, record it in run metadata, and upgrade deliberately.

DESIGN PATTERN LINK

Factory Method + versioned configuration: upgrades become explicit code changes.

55.1%

of projects: the most common smell (52.8% of all occurrences).

TNES: Temperature Not Explicitly Set

SMELLY

```
response = client.chat.completions.create(  
    model=MODEL,  
    messages=messages,  
    # temperature omitted -> provider default  
)
```

REFACTORED

```
response = client.chat.completions.create(  
    model=MODEL,  
    messages=messages,  
    temperature=0.0, # reproducible grading  
)
```

PROBLEM

Implicit defaults differ across providers, models and updates: results drift silently.

REFACTOR

Make the sampling policy explicit in code and document why that value was chosen.

DESIGN PATTERN LINK

Configuration Object / Factory: one place owns the sampling policy.

43.9%

of projects left temperature to the provider default.

UMM: Unbounded Max Metrics

SMELLY

```
client = OpenAI()
response = client.responses.create(
    model=MODEL, input=prompt)
# no output bound, no timeout, no retry budget
```

REFACTORED

```
client = OpenAI(timeout=30, max_retries=2)
response = client.responses.create(
    model=MODEL, input=prompt,
    max_output_tokens=512,
)
monitor_input_tokens(prompt)
```

PROBLEM

Runaway latency, cost and memory; one slow call can cascade into system-wide failure.

REFACTOR

Bound max output tokens, timeout and retries; monitor input size.

DESIGN PATTERN LINK

Gateway / Facade: operational policy is centralised and auditable.

42.1%

of projects had unbounded token, timeout or retry settings.

RENES: Reasoning Effort Not Explicitly Set

SMELLY

```
# reasoning-capable model
response = client.responses.create(
    model="o3-2025-04-16",
    input=prompt,
    # reasoning effort omitted
)
```

REFACTORED

```
response = client.responses.create(
    model="o3-2025-04-16",
    input=prompt,
    reasoning={"effort": "low"}, # fast FAQ path
)
```

PROBLEM

The accuracy / latency / cost trade-off is left implicit and invisible to reviewers.

REFACTOR

Explicitly configure reasoning effort wherever the model and API support it.

DESIGN PATTERN LINK

Strategy / Policy: the inference mode is a named, swappable choice.

0.9%

of projects: new, but growing with reasoning-capable models.

LLM Code Smells



Structural

[No System Message]

[Anonymous Inference Call]*

Defects in how the API request is composed.



Data Semantic

[No Structured Output]

[Raw Vision Payload]*

Incorrect assumptions about input/output structures.



Protocol

[Unbounded Max Metrics]

[LLM Temperature Not Explicitly Set]

[No Model Version Pinning]

[Reasoning Effort Not Explicitly Set]*

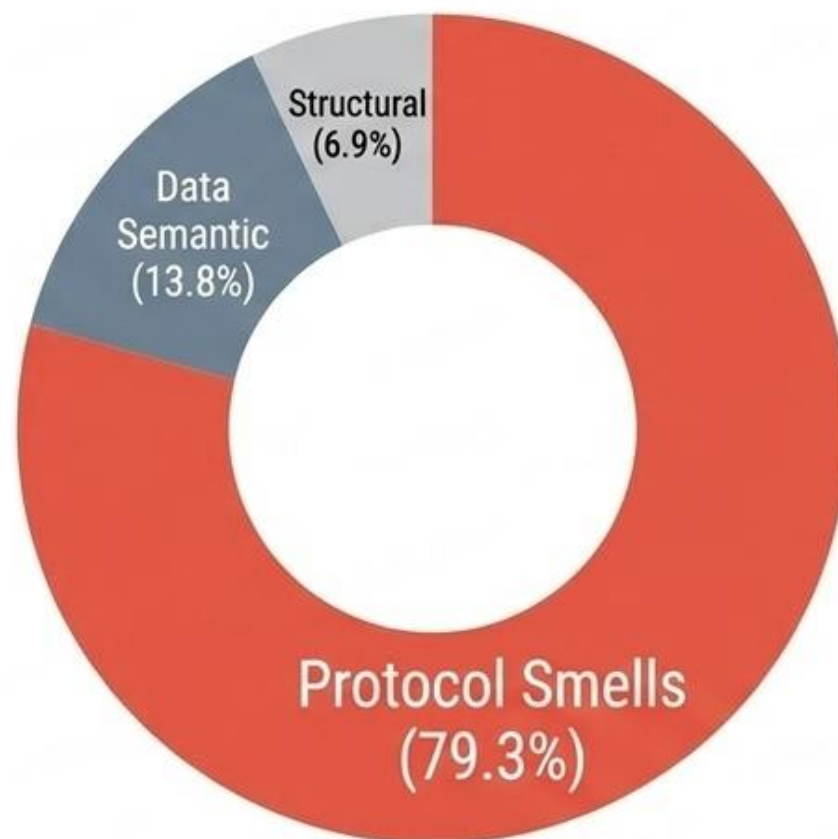
[Overspecified Sampling Parameters]*

Unstable system-wide inference configurations.

The Hidden Threat: Protocol Dominance

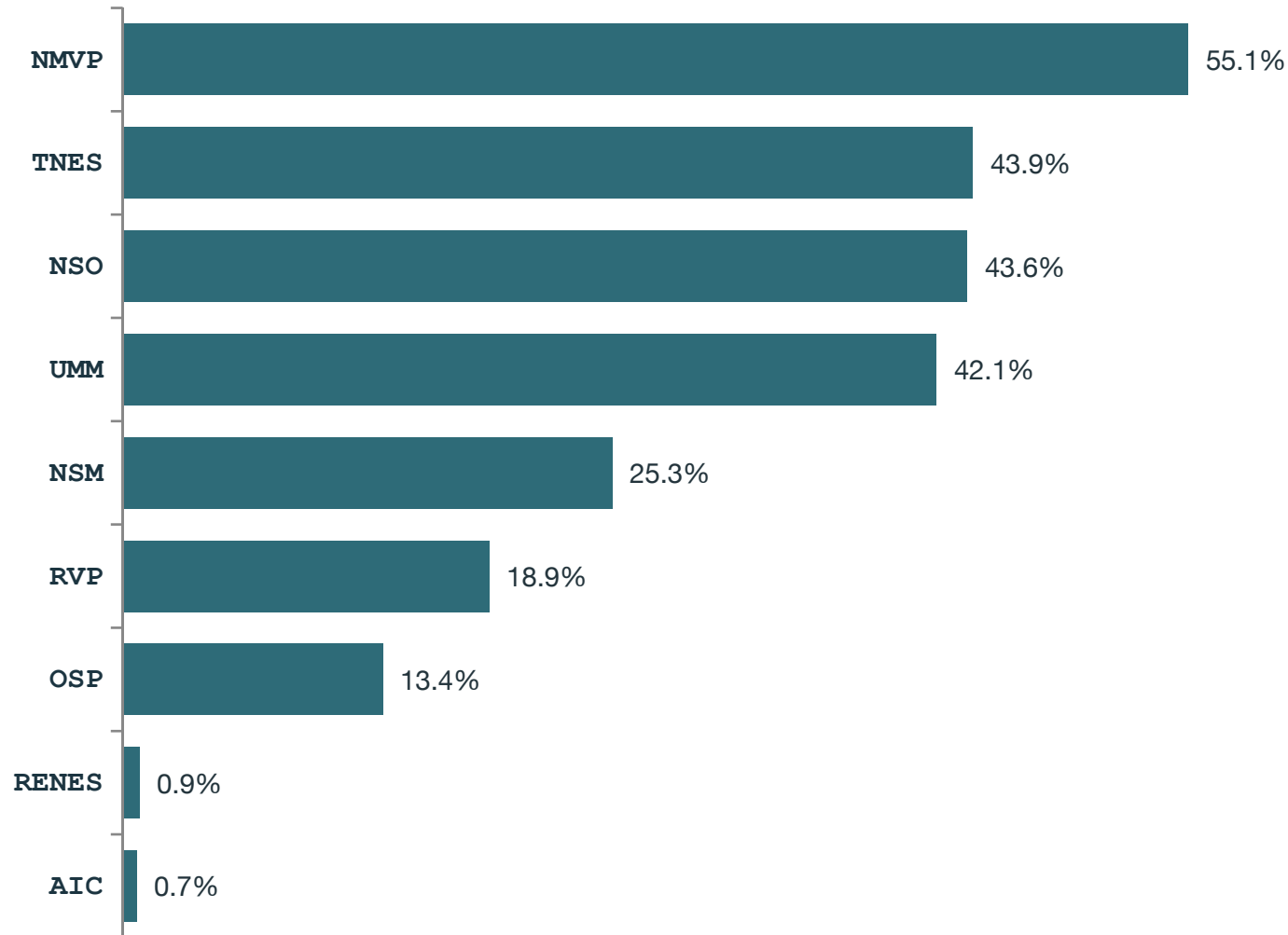
79.3%

of all detected smells
are Protocol-related.



Key Insight: In real-world software, the dominant integration risks aren't exotic API misuses. They are **missing explicitness**—failing to lock down moving versions, implicit defaults, and unbounded resource constraints.

How Common Are These Smells?



73.6%

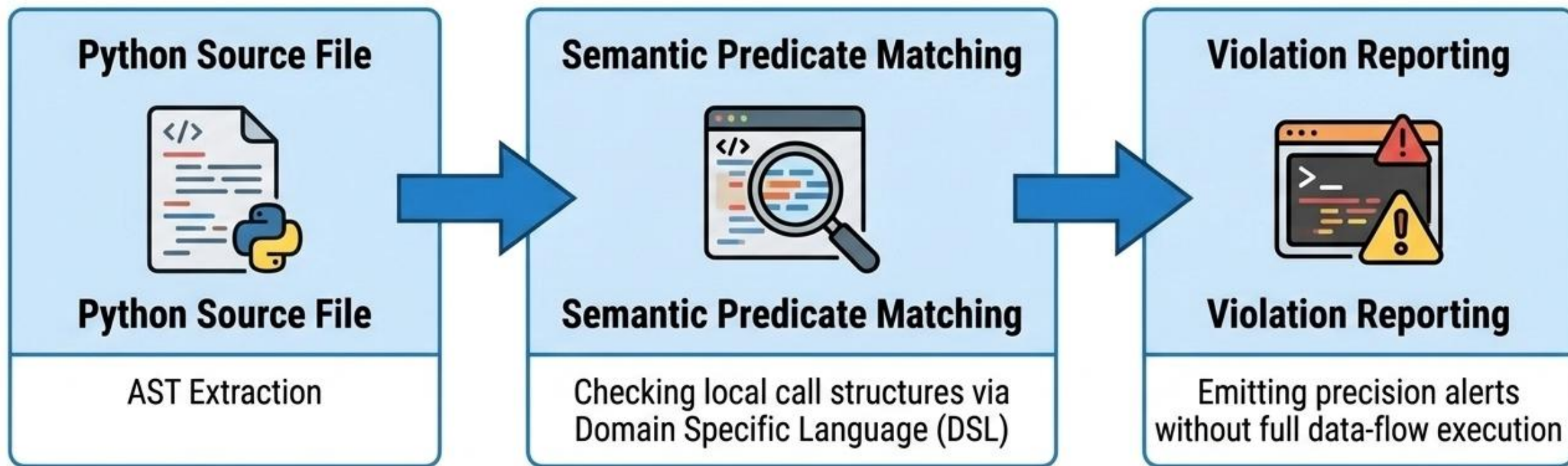
of projects contain at least one LLM code smell

PATTERN IN THE DATA

Smells are widespread across projects but concentrated in a small subset of files: the client factories, wrappers and orchestration modules.

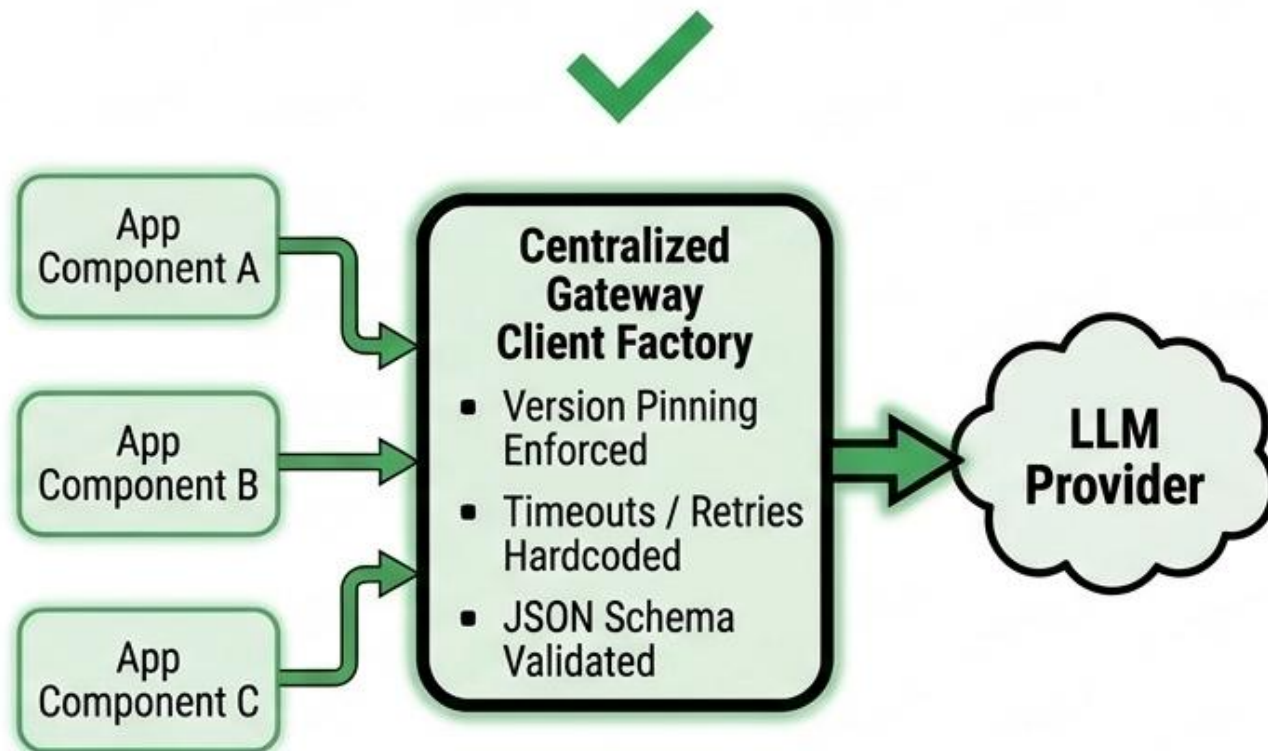
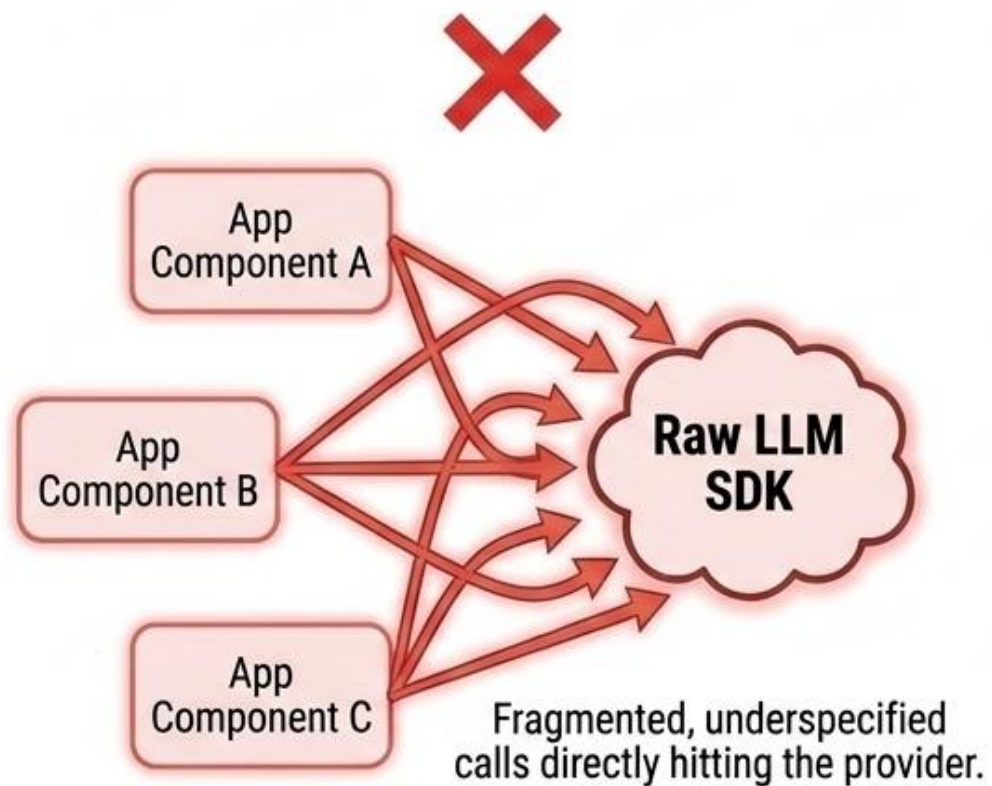
Automated Detection: SpecDetect4LLM

- Lightweight, intra-file static analysis prioritizing precision (91.3%) over aggressive recall.



Design Insight: Designed to be “soundy”—reporting violations only when explicitly supported by syntactic evidence at the call site to drastically reduce developer noise.

Architectural Mitigation: The Gateway Wrapper Pattern



Centralize the integration contract. Restricting direct provider calls to a single wrapper ensures configurations are explicit, auditable, and enforceable.

What Does the Evidence Suggest?

1

Protocol smells dominate

79.3% of detector occurrences were NMVP, TNES, UMM or RENES. The common theme: implicit defaults and unstable configuration.

2

Debt is localised

Smells cluster in client factories, wrappers and orchestration modules: not across the whole codebase. That makes them cheap to fix.

3

Centralise the policy

A single wrapper / client abstraction can enforce defaults, resource limits, logging, versioning and structured output at one auditable boundary.

Putting It Together: Before → After

BEFORE: A TYPICAL HELPER

```
def ask(prompt):
    client = OpenAI()
    r = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user",
                    "content": prompt}],
    )
    return r.choices[0].message
        .content.split(":") [1]

# NMVP  NSM  TNES  UMM  NSO  AIC
# six smells in nine lines
```



AFTER: ONE AUDITABLE BOUNDARY

```
class LLMGateway:
    # Facade
    MODEL = "gpt-4o-2024-11-20" # NMVP
    def __init__(self, client=None):
        self.client = client or make_client(
            timeout=30, max_retries=2) # Factory, UMM

    def ask(self, prompt, user_id, schema):
        r = self.client.chat.completions.parse(
            model=self.MODEL,
            messages=[{"role": "system", "content": POLICY},
                      {"role": "user", "content": prompt}], # NSM
            temperature=0.0, # TNES
            max_tokens=512, # UMM
            response_format=schema, # NSO
            user=pseudonymize(user_id), # AIC
        )
        return schema.from_response(r) # Adapter
```

LLM Code Smells × Design Patterns

Pattern	Intent (GoF)	What it enforces for LLM calls	Smells addressed
Facade / Gateway	One unified entry point to a subsystem	Bounds, logging, model policy, output validation at a single boundary	UMM, AIC, NSO, NSM
Factory Method	Centralise object creation	Pinned versions, provider configuration, consistent defaults for every client	NMVP, TNES, UMM
Strategy	Interchangeable algorithms behind one interface	Named sampling, reasoning-effort and vision-preprocessing policies	OSP, TNES, RENES, RVP
Adapter	Convert one interface into another	Provider-specific responses become stable, typed domain objects	NSO
Template Method	Fix the skeleton, vary the steps	Stable system instruction scaffold with a variable user part	NSM
Chain of Responsibility	Pass a request along a chain of handlers	Composable input-hygiene steps: resize → compress → redact → limit	RVP

A Smell-Resistant LLM Integration Architecture

Application / use cases

Chatbot, grader, summariser: never calls a provider directly

LLM Facade / Gateway

Single entry point: bounds, logging, validation, attribution

Policy & configuration

Pinned models, sampling & reasoning strategies, schemas, system policy

Provider adapters

OpenAI, Anthropic, local: translated into one domain interface

Observability & evaluation

Run metadata, traces, cost, regression tests in CI

RULE OF THUMB

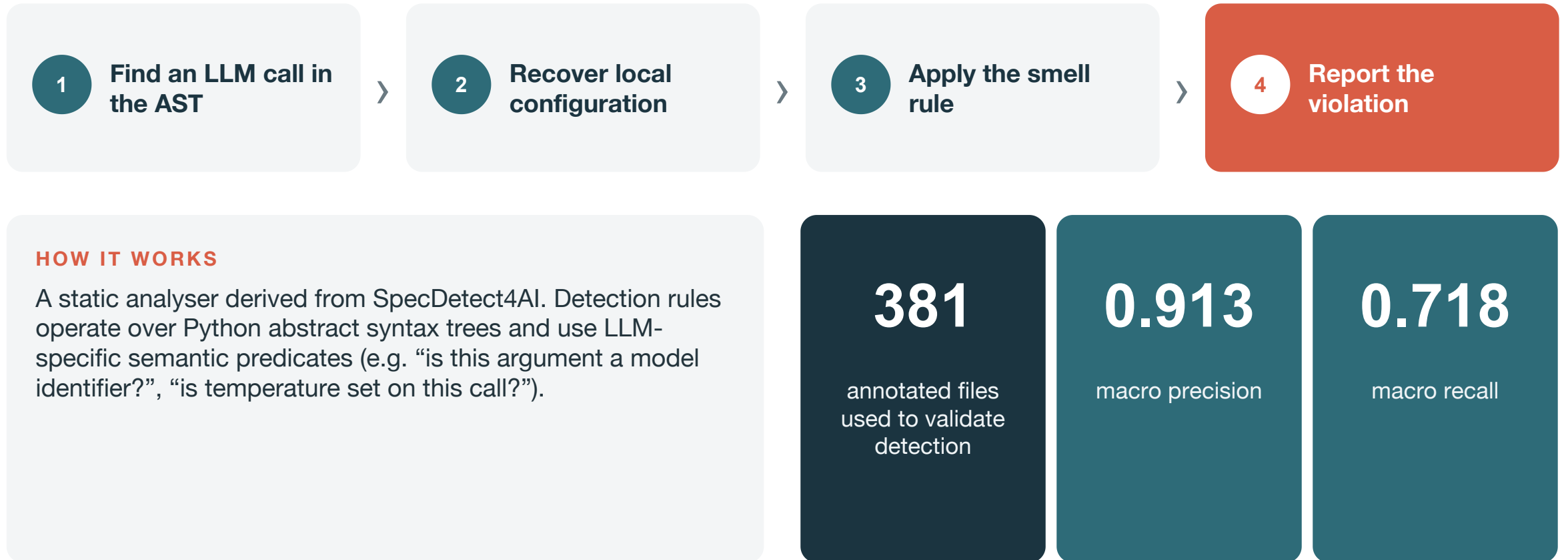
Application code should never see a provider SDK. If a smell can only be fixed in one file, that file is the gateway.

WHAT GOES WHERE

Wrapper: model, bounds, policy, validation, tracing.

Application: prompt content, domain logic, what to do with the typed result.

Can We Detect LLM Code Smells Automatically?



Your Next Assignment

HOW IT WORKS

Collect data from huggingface and
detect LLM Smells there

But Static Detection Has Limits

-  **Intra-file evidence only**
The analyser sees what is locally recoverable inside one file.
-  **Indirection hides constraints**
Cross-file wrappers, factories and runtime configuration are invisible to it.
-  **False positives**
Policy enforced elsewhere still looks like a missing parameter.
-  **False negatives**
Framework abstractions can hide a smelly call entirely.

THEREFORE

A tool warning is a review signal: not an automatic proof of a design defect.

Ironically, the Facade that fixes the smells is exactly what makes them harder to detect statically. Good architecture and good tooling must be designed together.

Smell or Good Design?

SCENARIO

A university help-desk chatbot uses one shared LLM client. Code review reveals:

- `model = "latest"`
- `no timeout`
- `no max output tokens`
- `temperature omitted`
- `raw user prompt only: no system message`
- `free-form reply parsed with split(':')`
- `every request includes session_id`

Task: name the smells, then propose a refactoring architecture.

DISCUSSION QUESTIONS

- 1 Which smell would you fix first, and why?
- 2 Which pattern centralises the most fixes?
- 3 What belongs in the wrapper vs. in application code?
- 4 Which of these properties should CI test for?

Hint: one item in the list is not a smell. Which one, and which smell does it prevent?

LLM Integration Code Review Checklist

☐

Is the model identifier pinned and recorded?

NMVP

☐

Are output tokens, timeout and retries bounded?

UMM

☐

Is the sampling policy explicit and minimal?

TNES · OSP

☐

Is reasoning effort explicit when applicable?

RENES

☐

Is there a stable system-level instruction?

NSM

☐

Is output structure enforced and validated?

NSO

☐

Are vision inputs preprocessed and size-bounded?

RVP

☐

Is user / session attribution propagated?

AIC

☐

Are provider calls centralised behind one auditable abstraction?

All

Summary

1

LLMs are components

Treat every LLM call as a software component with an explicit contract: model, instructions, output, sampling, limits, trace.

2

Smells are design signals

A smell may not be a bug: but it marks a recurring risk to robustness, performance, maintainability or reliability.

3

Patterns are the fix

Facade, Factory Method, Strategy, Adapter and Template Method make integration policy explicit, centralised and testable.

Good LLM engineering = **explicit behaviour + bounded resources + stable interfaces + observable execution**

References

1

Chenail-Larcher, Z., Mahmoudi, B., Moha, N., Stiévenart, Q., & Avellaneda, F. (2026). **LLM Code Smells: A Taxonomy and Detection Approach**. arXiv:2605.22976v1.: Nine-smell taxonomy, large-scale study of 692 projects, SpecDetect4LLM.

2

Mahmoudi, B., Chenail-Larcher, Z., Moha, N., Stiévenart, Q., & Avellaneda, F. (2026). **Specification and Detection of LLM Code Smells**. ICSE-NIER '26.: Earlier paper introducing the first five smells and the specification approach.

3

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). **Design Patterns: Elements of Reusable Object-Oriented Software**. Addison-Wesley.: Facade, Factory Method, Strategy, Adapter, Template Method, Chain of Responsibility.

All prevalence figures, precision/recall values and smell definitions on these slides are taken from the first paper.